An Overview of AFL

(american fuzzy lop)

Claire Shin

Introduction

What is Fuzzing?

What is Fuzzing?

- Automated software crash-testing
- Done by feeding in fuzz to a program
  - Random, unexpected, or invalid data
- Goal: cause the program to crash, freeze, etc.

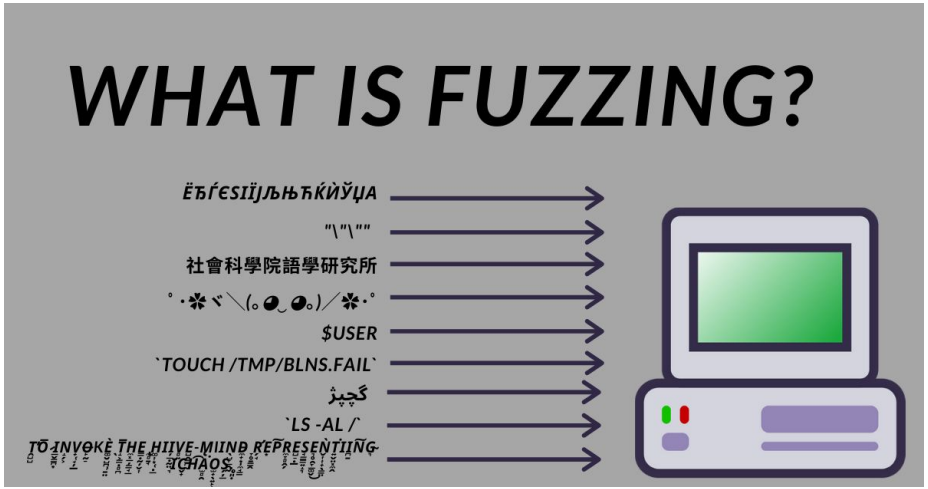


Image Credit: Kealan Parr

The (Historical) Problem

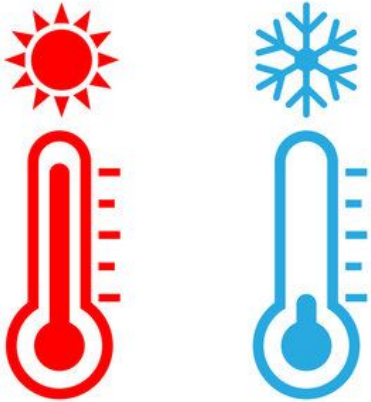
1. Blind Random Mutation

- Fuzzer takes a valid file, randomly mutates it and feeds it to the program
- Input validation checks/other guardrails in the code make it less effective
- Fast but not very smart

2. Symbolic Execution

- Uses mathematics to analyze code, calculating inputs to reach inner logic of program
- Very slow and computationally intensive
- Smart but very expensive (especially on real software)

So… how to solve this?



The Solution: AFL

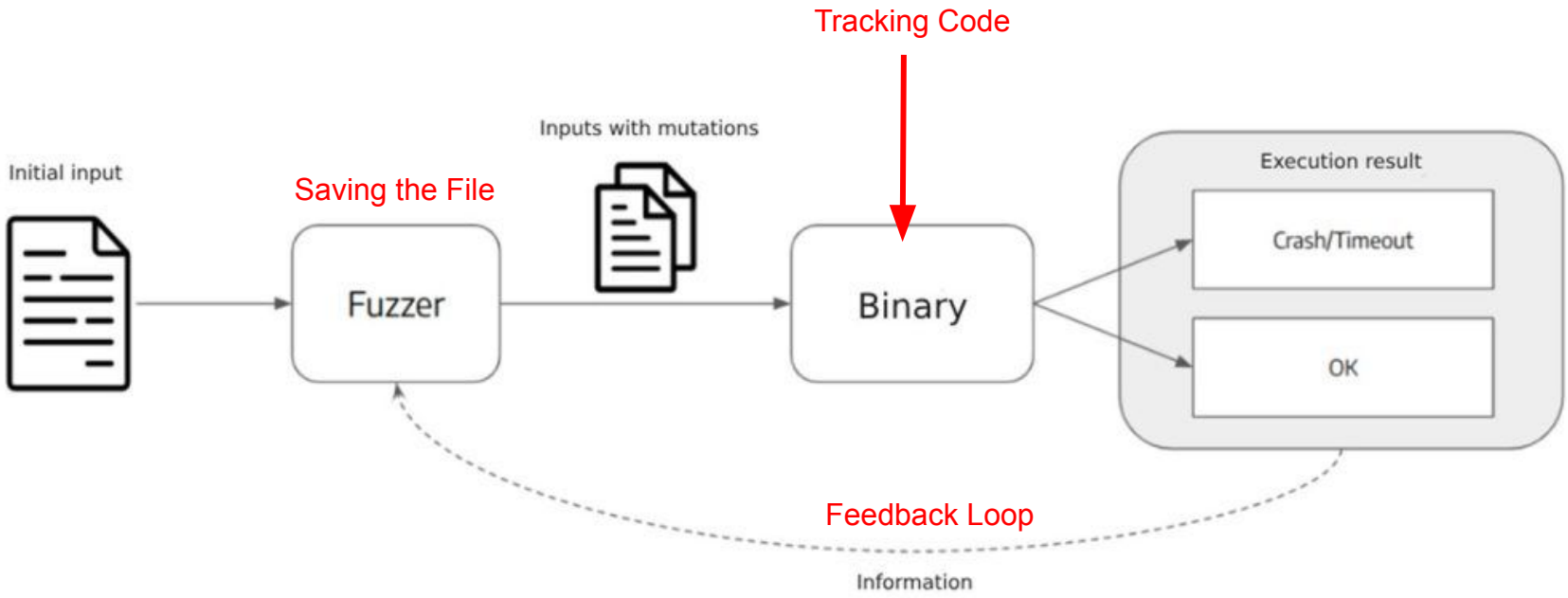


Image Credit: André Bermudes Viana (with edits from me)

AFL’s Design Goals

Speed

Reliability

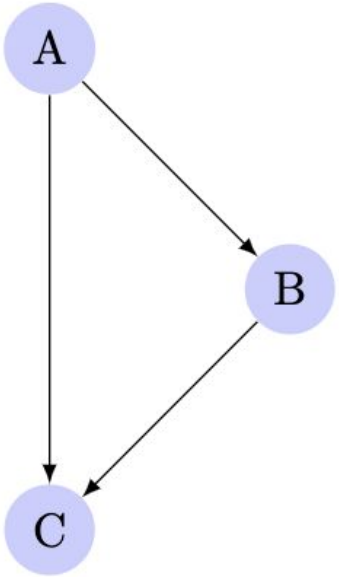
Simplicity

Chainability

The Secret Sauce

1. Edge Coverage

- Block vs. Edge
  - A -> B is different from B -> A
- 64KB (shared) memory map tracks every unique edge (edge coverage)
  - Set byte is a hit for a branch in (src, dst) form
- XOR & bit shifting
  - Used to calculate paths quickly
  - ID of A + ID of B = unique path ID



2a. The Evolutionary Algorithm

- The non-greedy approach
  - Multiple “interesting” files
- Smart mutations
  - Deterministic bit flips, math, file splicing
- Fitness test (survival of the fittest)
  - Only mutations that trigger new edges or loop behaviors “survive”
- Final corpus: interesting test cases
- Execution is policed via memory and execution time limits

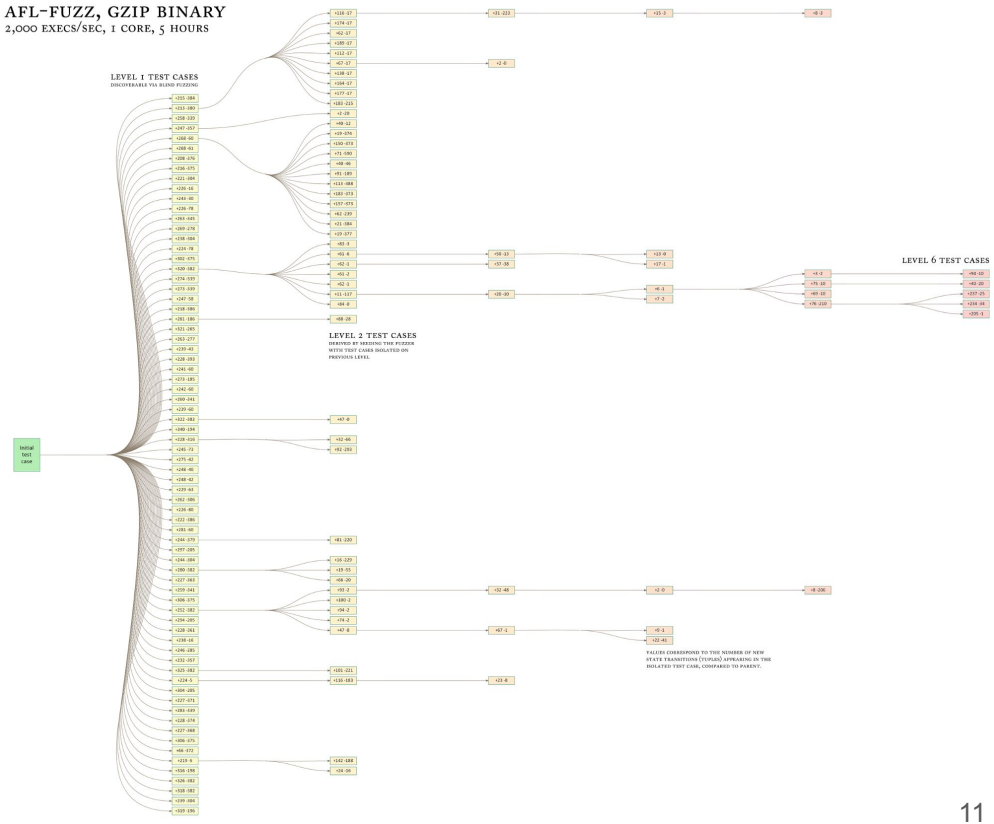


Image Credit: educba.com

2b. The Evolutionary Algorithm

Culling the Corpus:

- Algorithm saves every file that triggers a new path
- Over time:
  - Old file: [A, B]
  - Newer file: [A, B, C, D]
  - Overlap - wastes CPU time
- Solution: Periodically re-evaluate the entire queue to pick “favorites” (fastest/smallest)

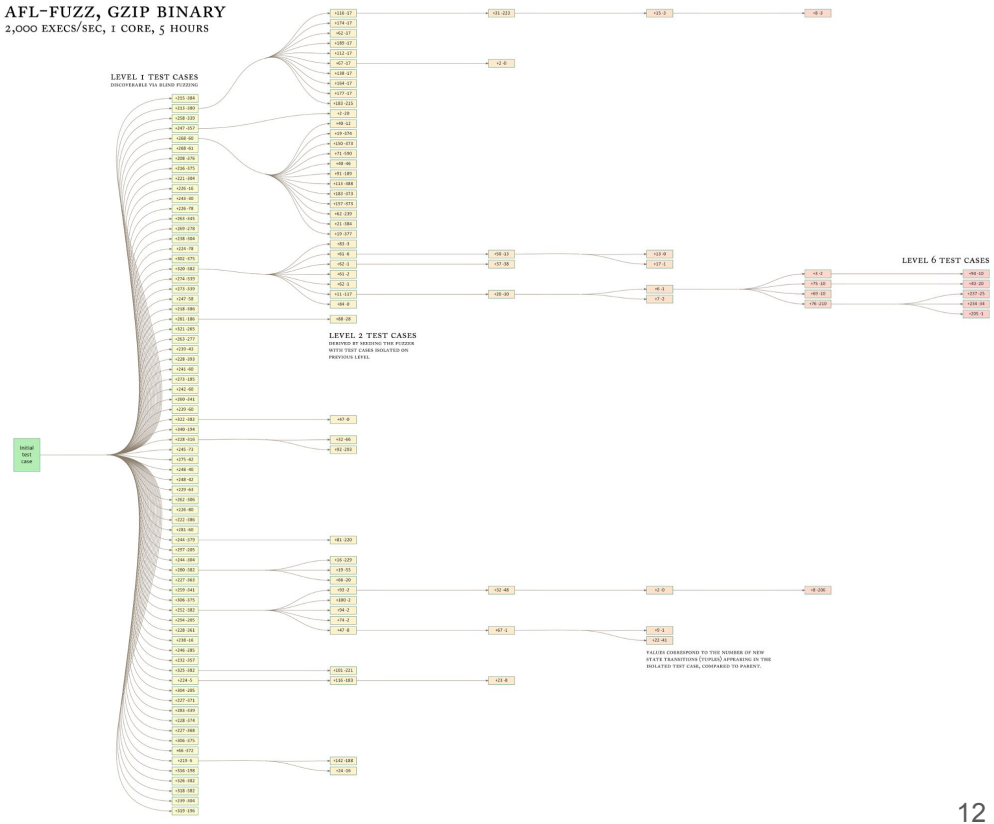


Image Credit: educba.com

3. Mutation Strategies

The Progression: Deterministic -> Non-deterministic

- Starts with bit flips and integers to produce compact crashes
- Progresses into non-deterministic stacked mutations and file splicing

Effector Maps

- If a file region doesn't affect control flow, AFL skips fuzzing it
- Reduces execution time

Dictionaries

- Uses feedback to auto-detect and combine syntax tokens
- Organically reconstructs complex languages (JavaScript, SQL, XML)

4. Crash Management & Exploration

De-duplicating Crashes

- Avoids naive methods like grouping by memory address
- Crash is unique if it triggers a new edge

Crash Exploration

- Fuzzes a crashing file but throws away non-crashing mutations
- Helps us figure out whether a bug can be exploited

No Auto-Trimming

- Unlike the rest of the queue, crash files are never trimmed
- Preserves bytes needed to compare against its parent (that doesn’t crash)

5. Trimming & the Fork Server

- Fork Server: execution speed
  - Clones a paused process instead of rebooting from scratch
  - Bypasses OS overhead (linking, libc initialization)
- Forking Modes
  - Deferred mode: skips over larger, user selected chunks of initialization code
  - Persistent mode: tests multiple inputs in a single process
- File Trimming: input speed
  - Trim dead weight from files
  - Smaller files = faster processing

6. Parallelization & Black-Box Testing

Parallelization

- Run independent instances across multiple CPU cores or remote machines
- Instances periodically sync, sharing only test cases that trigger new paths
- Synergy: Can test different parsers of the same format simultaneously

Black-Box Testing (No Source Code)

- Uses QEMU emulation to instrument compiled binaries
- Cross-architecture support (e.g., fuzzing ARM code on an x86 machine)

Emulation Optimized for Speed

- Uses the Fork Server and translation caching to bypass slow emulator startup times

```
                    american fuzzy lop 1.86b (test)

┌─ process timing ─────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 0 min, 2 sec        │  cycles done : 0      │
│   last new path : none seen yet                      │  total paths : 1      │
│ last uniq crash : 0 days, 0 hrs, 0 min, 2 sec        │ uniq crashes : 1      │
│  last uniq hang : none seen yet                      │   uniq hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 0 (0.00%)         │    map density : 2 (0.00%)             │
│ paths timed out : 0 (0.00%)         │ count coverage : 1.00 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : havoc                 │ favored paths : 1 (100.00%)            │
│ stage execs : 1464/5000 (29.28%)    │  new edges on : 1 (100.00%)            │
│ total execs : 1697                  │ total crashes : 39 (1 unique)          │
│  exec speed : 626.5/sec             │   total hangs : 0 (0 unique)           │
├─ fuzzing strategy yields ───────────┴───────────────┬─ path geometry ────────┤
│   bit flips : 0/16, 1/15, 0/13                      │    levels : 1          │
│  byte flips : 0/2, 0/1, 0/0                         │   pending : 1          │
│ arithmetics : 0/112, 0/25, 0/0                      │  pend fav : 1          │
│  known ints : 0/10, 0/28, 0/0                       │ own finds : 0          │
│  dictionary : 0/0, 0/0, 0/0                         │  imported : n/a        │
│       havoc : 0/0, 0/0                              │  variable : 0          │
│        trim : n/a, 0.00%                            ├────────────────────────┘
└─────────────────────────────────────────────────────┘          [cpu: 92%]
```

Limitations

Where AFL Struggles

- if (input == “SECRET_KEY”)
  - The odds of AFL randomly mutating a file to spell that string are basically 0
- 64KB map collision
  - Every path is tracked in a memory map -> eventually, overlaps
- Structured file formats (JSON, JS, etc.)
  - Wasted cycles generating syntax errors
- Power scheduling
  - Treats all interesting files “fairly” -> time wasted on regular errors

Improved Versions (Modern Fuzzers)

AFL++

- Hybrid fuzzing
- Collision-free tracking
- Grammar-aware fuzzing
- Smart power scheduling

Conclusions

Actionable Takeaways

1. Go beyond unit tests: integrate fuzzing into CI/CD pipelines
2. Fuzzer -> Target: match the right kind of fuzzer to a target
3. Speed > Perfection: practicality beats elegance (most of the time)